

# Python Básico

## IV. Estruturas

### Estruturas de repetição

# Conclusões - Estruturas condicionais:

- Relação de estruturas condicionais com os programas de computador
- Relação de estruturas condicionais com fluxogramas
- Estrutura condicional e bloco de código
- O que são blocos de código
- Como determinar um bloco de código





# Objetos

Python



```
1 escalar = 1
```

[3] ✓ 0.0s



```
1 vetor = [1, 2, 3, 4, 5]
```

[4] ✓ 0.0s



```
1 matriz = [[1, 2], [3, 4], [5, 6]]
```

[5] ✓ 0.0s



# Iteração

Repetição de uma ou mais ações.

Python



```
1 obj_tupla_int = (1, 0, 1, 0)
2 obj_tupla_bool = (True, False, True, False)
3 print(f"{obj_tupla_int} == {obj_tupla_bool}: {obj_tupla_int == obj_tupla_bool}")
4 print(f"{obj_tupla_int} != {obj_tupla_bool}: {obj_tupla_int != obj_tupla_bool}")
5 print(f"{obj_tupla_int} > {obj_tupla_bool}: {obj_tupla_int > obj_tupla_bool}")
6 print(f"{obj_tupla_int} >= {obj_tupla_bool}: {obj_tupla_int >= obj_tupla_bool}")
7 print(f"{obj_tupla_int} < {obj_tupla_bool}: {obj_tupla_int < obj_tupla_bool}")
8 print(f"{obj_tupla_int} <= {obj_tupla_bool}: {obj_tupla_int <= obj_tupla_bool}")
```

[10]

✓ 0.0s

...

```
(1, 0, 1, 0) == (True, False, True, False): True
(1, 0, 1, 0) != (True, False, True, False): False
(1, 0, 1, 0) > (True, False, True, False): False
(1, 0, 1, 0) >= (True, False, True, False): False
(1, 0, 1, 0) < (True, False, True, False): False
(1, 0, 1, 0) <= (True, False, True, False): False
```

# Iteração

Repetição de uma ou mais ações.

Enquanto for verdade:

Faça alguma coisa

Iteração

Python



# Iteração

```
1 a = 4
2 b = 2
3 operadores = [
4     ['+', a + b],
5     ['-', a - b],
6     ['*', a * b],
7     ['/', a / b],
8     ['%', a % b],
9     ['**', a ** b],
10    ['//', a // b],
11    ['==', a == b],
12    ['!=', a != b],
13    ['>', a > b],
14    ['<', a < b],
15    ['>=', a >= b],
16    ['<=', a <= b],
17    ['and', a > b and b > a],
18    ['or', a > b or b > a],
19 ]
```

Enquanto for verdade:  
Faça alguma coisa

Iteração



```
20 c = 0
21 print(f"{a} {operadores[c][0]} {b} = {operadores[c][1]}")
22 c += 1
23 print(f"{a} {operadores[c][0]} {b} = {operadores[c][1]}")
24 c += 1
25 print(f"{a} {operadores[c][0]} {b} = {operadores[c][1]}")
26 c += 1
27 print(f"{a} {operadores[c][0]} {b} = {operadores[c][1]}")
28 c += 1
29 print(f"{a} {operadores[c][0]} {b} = {operadores[c][1]}")
30 c += 1
31 print(f"{a} {operadores[c][0]} {b} = {operadores[c][1]}")
32 c += 1
33 print(f"{a} {operadores[c][0]} {b} = {operadores[c][1]}")
34 c += 1
35 print(f"{a} {operadores[c][0]} {b} = {operadores[c][1]}")
36 c += 1
37 print(f"{a} {operadores[c][0]} {b} = {operadores[c][1]}")
38 c += 1
39 print(f"{a} {operadores[c][0]} {b} = {operadores[c][1]}")
40 c += 1
41 print(f"{a} {operadores[c][0]} {b} = {operadores[c][1]}")
42 c += 1
43 print(f"{a} {operadores[c][0]} {b} = {operadores[c][1]}")
44 c += 1
45 print(f"{a} {operadores[c][0]} {b} = {operadores[c][1]}")
46 c += 1
47 print(f"{a} {operadores[c][0]} {b} = {operadores[c][1]}")
48 c += 1
49 print(f"{a} {operadores[c][0]} {b} = {operadores[c][1]}")
```

[14] ✓ 0.0s

```
4 + 2 = 6
4 - 2 = 2
4 * 2 = 8
4 / 2 = 2.0
4 % 2 = 0
4 ** 2 = 16
4 // 2 = 2
4 == 2 = False
4 != 2 = True
4 > 2 = True
4 < 2 = False
4 >= 2 = True
4 <= 2 = False
4 and 2 = False
4 or 2 = True
```

Python





# Iteração

Python



```
1 a = 4
2 b = 2
3 operadores = [
4     ['+', a + b],
5     ['-', a - b],
6     ['*', a * b],
7     ['/', a / b],
8     ['%', a % b],
9     ['**', a ** b],
10    ['//', a // b],
11    ['==', a == b],
12    ['!=', a != b],
13    ['>', a > b],
14    ['<', a < b],
15    ['>=', a >= b],
16    ['<=', a <= b],
17    ['and', a > b and b > a],
18    ['or', a > b or b > a],
19 ]
```

```
1 for operador in operadores:
2     print(f"{a} {operador[0]} {b} = {operador[1]}")
```

[15] ✓ 0.0s

```
... 4 + 2 = 6
     4 - 2 = 2
     4 * 2 = 8
     4 / 2 = 2.0
     4 % 2 = 0
     4 ** 2 = 16
     4 // 2 = 2
     4 == 2 = False
     4 != 2 = True
     4 > 2 = True
     4 < 2 = False
     4 >= 2 = True
     4 <= 2 = False
     4 and 2 = False
     4 or 2 = True
```

Enquanto for verdade:  
Faça alguma coisa

Iteração



# Iteração

Python



```
1 vetor_num = [10, 20, 30, 40, 50, 60]
2 vetor_num

[16] ✓ 0.0s

... [10, 20, 30, 40, 50, 60]
```

```
1 vetor_num = []
2 contador = 10
3 while contador <= 60:
4     vetor_num.append(contador)
5     contador += 10
6 vetor_num

[22] ✓ 0.0s

... [10, 20, 30, 40, 50, 60]
```

Enquanto for verdade:  
Faça alguma coisa

Iteração



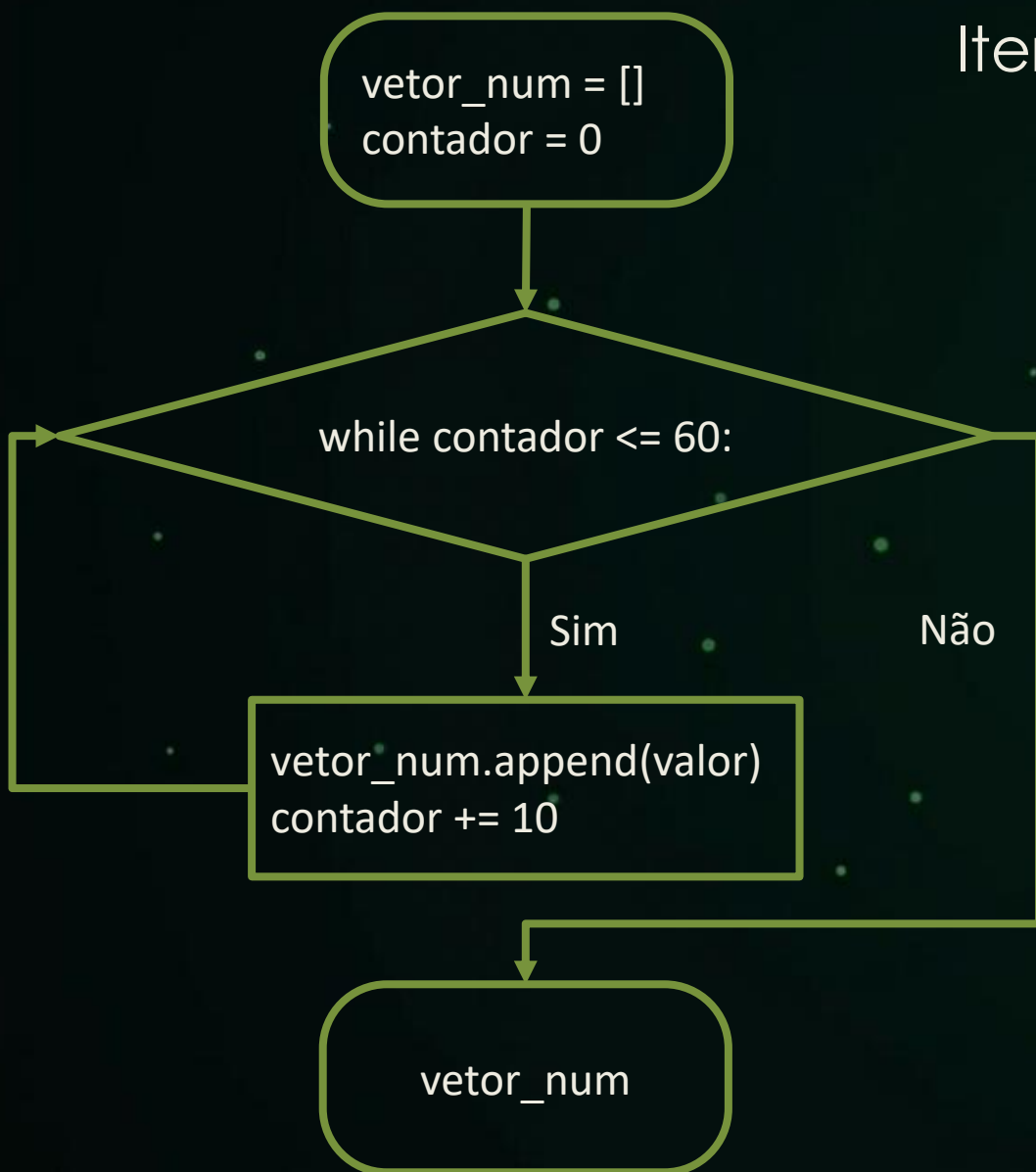
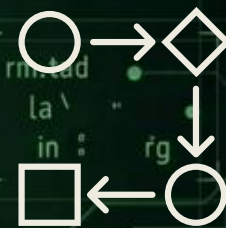


# Estruturas de repetição

Enquanto for verdade:  
Faça alguma coisa

# Iteração

## Endentação (tab)



▷ ✓

```
1  vetor_num = []
2  contador = 10
3  while contador <= 60:
4      |    vetor_num.append(contador)
5      |    contador += 10
6  vetor_num
```

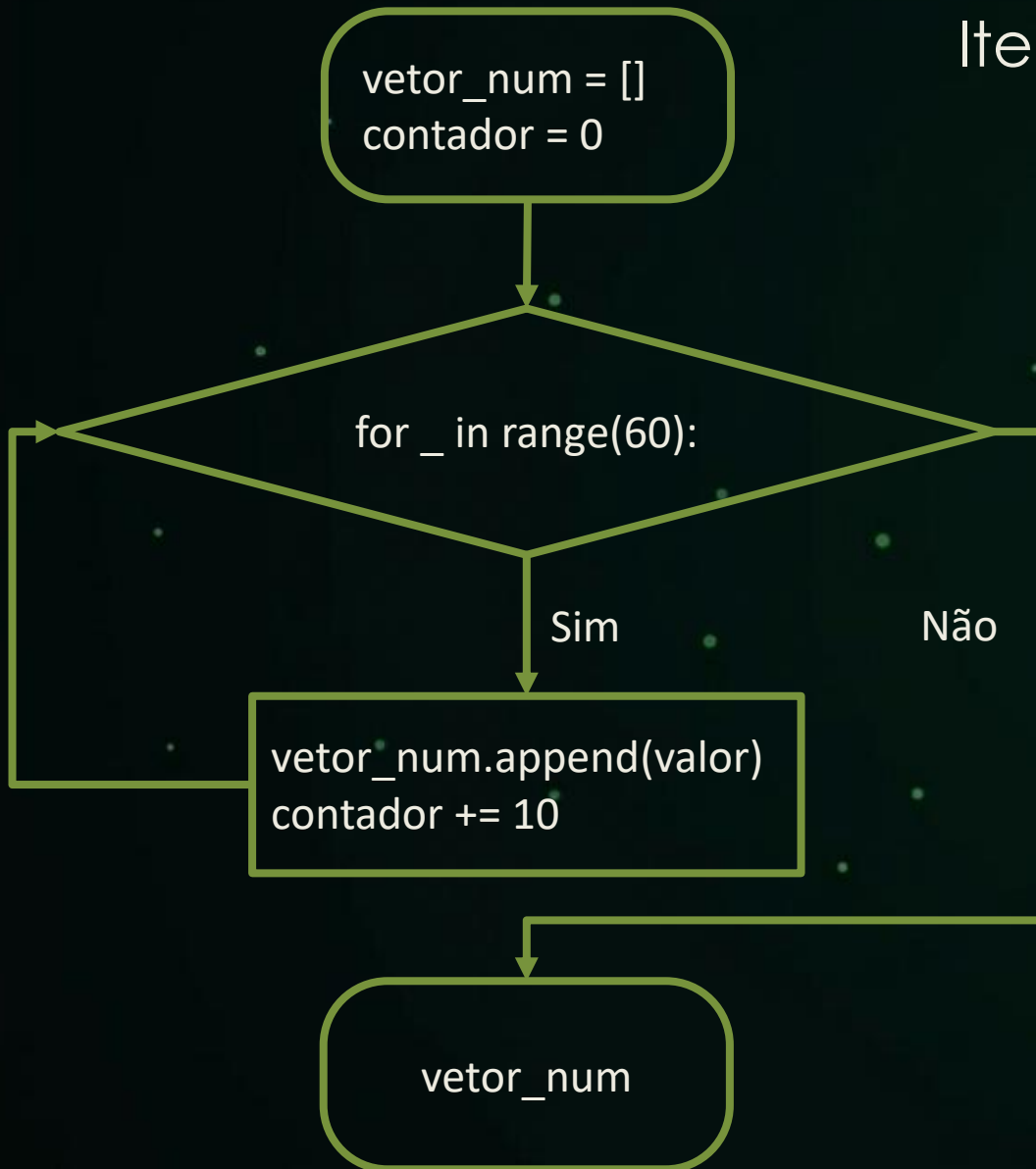
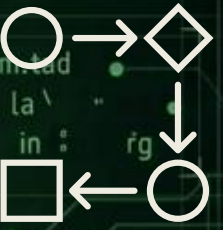
|      |   |      |
|------|---|------|
| [22] | ✓ | 0.0s |
|------|---|------|

```
... [10, 20, 30, 40, 50, 60]
```

# Estruturas de repetição

Enquanto for verdade:  
Faça alguma coisa  
Iteração

Endentação (tab)



```
1  vetor_num = []
2  contador = 10
3  for _ in range(6):
4      vetor_num.append(contador)
5      contador += 10
6  vetor_num
```

[23] ✓ 0.0s

... [10, 20, 30, 40, 50, 60]

# Conclusões

- Objetos
- Iteração
- for
- while

Python







Vamos  
praticar?

LIMC-IA



Vamos  
exercitar?

LIMC-1A



## Exercícios:

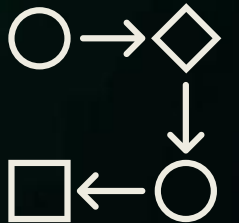
Instancie um objeto da classe lista contendo 5.000 valores inteiros iniciando em 0.

Quantos são pares e quantos são ímpares?

01 – Descreva o racional da sua resposta:

02 – Desenhe o fluxograma da resposta:

03 – Escreva o programa em Python:



## Exercícios:

04 – Substitua a estrutura de repetição e responda novamente a pergunta anterior.





## Exercícios:

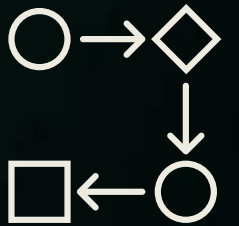
Importe o .csv exportado na última aula e instancie um objeto da classe DataFrame.

Dos números de letras nos genes, quais são pares e quantos são ímpares?

05 – Descreva o racional da sua resposta:

06 – Desenhe o fluxograma da resposta:

07 – Escreva o programa em Python:



## Exercícios:

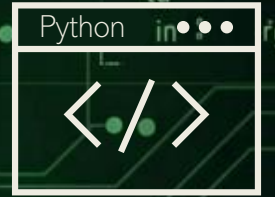
08 – Substitua a estrutura de repetição e responda novamente a pergunta anterior.



## Exercícios:

09 – Usando a coluna de presença de hu no nome do gene, itere os index deste objeto usando a função `enumerate()`. Estes index podem ser usados para imprimir cada um dos valores presentes na coluna dos dois primeiros caracteres dos nomes dos genes?

10 – Selecione qualquer uma das instâncias contidas na DataFrame. É possível recuperar as chaves e os valores com ao menos 3 estruturas de repetição diferentes?





# Python Básico



3 de abril de 2026